SMART CAMPUS

Complaint & Grievance Management System

Product and Engineering Specification Pack

Includes: Product Requirements (PRD) · Technical Requirements (TRD) · App Flow · UI/UX Design Brief · Backend Schema · Implementation Plan

Document control	Value
Version	1.0 — Draft for hackathon prototype
Prepared for	REVA Weekend Hackathon / campus use case
Product type	Responsive web application
Proposed stack	Flask + Jinja/HTML/CSS/JavaScript + PostgreSQL
Design direction	Apple-inspired: clarity, hierarchy, consistency, accessibility; not an official Apple document

Scope note: The core requirements are based on Problem Statement #3 in the supplied REVA Weekend Hackathon document (pages 3–4). Product roles, detailed security controls, schema fields and delivery phases below are implementation recommendations to make those requirements buildable.

Contents

- 1. Product Requirements Document (PRD)
- 2. Technical Requirements Document (TRD)
- 3. Application Flow
- 4. UI/UX Design Document — Apple-inspired principles
- 5. Backend and Database Schema
- 6. API / Route Contract
- 7. Implementation Plan
- 8. Acceptance Criteria and Test Plan
- 9. Risks, Assumptions and Decisions

1. Product Requirements Document (PRD)

1.1 Product summary

The Smart Campus Complaint and Grievance Management System is a web application for reporting campus issues and tracking them through to a recorded outcome. It replaces informal, hard-to-track reporting with a ticket-based workflow, department ownership, status history, feedback and basic resolution analytics.

1.2 Problem statement

Campus issues such as broken infrastructure, Wi-Fi failures, cleanliness concerns and transport problems may be reported through scattered channels. Reporters may not know who owns the issue or whether action is being taken. Departments lack a single queue and management lacks consistent resolution metrics.

1.3 Goals

- Let students and staff submit a complaint with description, category, location and optional image.
- Generate a unique, human-readable complaint ticket ID automatically.
- Route each complaint to an appropriate department, with authorized manual reassignment.
- Show the reporter the current status and chronological updates.
- Let authorized officers document work, resolution notes and rejection reasons.
- Collect post-resolution feedback and provide basic resolution-time analytics.

1.4 Non-goals for the first release

- AI-based complaint classification or automatic image understanding.
- Native iOS/Android applications; the first release is a responsive website.
- Public discussion threads, anonymous complaints, legal case management or emergency dispatch.
- Complex SLA escalation workflows, SMS gateways and multi-campus tenancy.

1.5 Users and roles

Role	Primary needs	Permissions
Student / Staff	Submit and follow up on own complaints	Create complaints; view own tickets and history; submit feedback after resolution
Department officer	Work a departmental queue	View department-assigned complaints; add remarks; update status; record resolution or rejection reason
Administrator	Oversee operations and accountability	View all complaints; manage categories/departments/officers; assign/reassign; access analytics

1.6 Functional requirements

ID	Requirement	Priority	Acceptance summary
FR-01	Authentication and role access	Must	Users log in; restricted routes enforce server-side permissions.
FR-02	Submit complaint	Must	Valid title, category, description and location create a saved ticket.
FR-03	Image attachment	Must	Optional image upload is type/size validated and stored safely.
FR-04	Unique ticket ID	Must	Every saved complaint has one unique ticket ID.
FR-05	Category and department	Must	Configured categories map to

	routing		a default department.
FR-06	Complaint list and search	Must	Users see only permitted tickets and can filter by status/category.
FR-07	Status tracking	Must	Authorized status changes update current state and append history.
FR-08	Resolution / rejection details	Must	Resolved tickets record resolution notes; rejected tickets require a reason.
FR-09	Feedback	Should	Reporter can submit one 1–5 rating and optional comment after resolution.
FR-10	Admin analytics	Must	Counts, category breakdown and resolution-time metrics use database data.
FR-11	Notifications	Could	In-app notification or email on submission and status changes.

1.7 Status model

Status	Meaning	Rules
Submitted	Ticket received	Initial state; ticket ID and submission timestamp are recorded.
In Progress	Department is working on it	Authorized officer/admin changes status; history entry required.
Resolved	Issue is reported as fixed	Resolution notes required; resolved_at timestamp set.
Rejected	Request is not accepted for action	Rejection reason required; reporter can view the reason.

Recommended transition rules: Submitted → In Progress, Resolved or Rejected; In Progress → Resolved or Rejected. Reopening is out of scope for v1 unless explicitly added. Any transition must be validated on the server.

1.8 Success metrics

- 100% of successfully created complaints receive a unique ticket ID.
- Every status change has actor, timestamp and resulting status in history.
- Unauthorized cross-user and cross-department access attempts are denied.
- Admin can calculate total, open, resolved and rejected counts from live data.
- Resolution-time metric is calculated from created_at to resolved_at for resolved tickets only.

1.9 MVP definition

MVP is complete when a demo user can log in, submit a complaint with an image, receive a ticket ID, see the department assignment, and track a real status update performed by an authorized officer; an administrator can inspect the ticket and see data-driven summary metrics.

2. Technical Requirements Document (TRD)

2.1 Proposed architecture

Use a server-rendered Flask application with Jinja templates and progressive JavaScript. PostgreSQL is the source of truth. Uploaded images are stored in private object storage or a protected local directory during development; the database stores a storage key/path rather than raw image bytes.

Layer	Technology	Responsibility
Presentation	HTML5, CSS, vanilla JavaScript, Jinja	Responsive pages, forms, status timeline, charts
Web application	Python + Flask	Routing, validation, sessions, authorization and workflow
ORM / migrations	Flask-SQLAlchemy + Flask-Migrate	Database access, schema versioning and migrations
Database	PostgreSQL	Users, departments, complaints, history, feedback
File storage	Private object storage or protected local uploads	Complaint image attachments
Charts	Chart.js	Admin dashboard visualizations
Testing	pytest + Flask test client; Postman optional	Unit, integration and route checks
Version control	Git + GitHub	Branches, code review and team integration

2.2 Non-functional requirements

Area	Requirement
Security	Password hashes; CSRF protection; secure session cookie; server-side role and object authorization; parameterized ORM queries.
Privacy	Users see only permitted complaint records and attachments; avoid public URLs for private images.
Reliability	Use transactions for ticket creation and status updates; roll back on failure.
Performance	Paginate complaint lists; index frequently filtered fields; avoid loading all records for dashboards.
Accessibility	Keyboard-accessible forms, visible focus, labels, meaningful errors and sufficient contrast.
Responsive design	Usable at mobile widths, tablet and desktop; tables adapt to cards or horizontal scrolling.
Maintainability	Use environment configuration, migrations, modular route/service code and automated tests.
Observability	Log unexpected errors and key workflow failures without logging passwords or sensitive complaint text unnecessarily.

2.3 Suggested dependencies

Flask, Flask-SQLAlchemy, Flask-Migrate, Flask-Login, Flask-WTF, python-dotenv, psycopg, Werkzeug, pytest and Chart.js. Pin tested versions in requirements.txt. The exact versions should be chosen and validated when implementation begins.

2.4 Configuration

- SECRET_KEY: long random secret supplied through environment variables.
- DATABASE_URL: PostgreSQL connection string supplied through environment variables.
- UPLOAD_FOLDER or object-storage credentials: never hard-code credentials in source.
- MAX_CONTENT_LENGTH: enforce a documented maximum upload size.
- ALLOWED_IMAGE_TYPES: allow only necessary raster image formats; verify file content, not just extension.
- DEBUG: enabled only in local development; disabled in deployed environments.

2.5 Image upload controls

- Allow only JPEG, PNG and WebP unless the project has a justified need for more formats.
- Set a small prototype limit, such as 5 MB per image, and enforce it server-side.
- Generate storage names server-side; do not use a submitted filename as a filesystem path.

- Store files outside executable/static public directories or use private object storage.
- Check upload success and clean up orphaned objects if the database transaction fails.

3. Application Flow

3.1 Primary end-to-end flow

Step	Actor	Action	System result
1	Student/staff	Log in	Credentials verified; role-specific dashboard opens.
2	Reporter	Choose New Complaint	Form is displayed.
3	Reporter	Enter details and optionally attach image	Client and server validation run.
4	System	Save complaint	Database ID created; ticket ID assigned; status set to Submitted.
5	System	Map category to department	Department assigned; initial history event recorded.
6	Reporter	Open ticket detail	Ticket ID, current status, department and history are visible.
7	Officer/admin	Review queue and update status	Permission and transition checked; history appended.
8	Officer	Resolve or reject	Resolution notes or rejection reason recorded; timestamp set where applicable.
9	Reporter	View outcome and submit feedback	Feedback is saved once for a resolved complaint.
10	Admin	Review dashboard	Live counts and resolution analytics are displayed.

3.2 Mermaid flowchart

The following flow can be copied into a Mermaid-compatible editor:

```
flowchart TD
  A[Open application] --> B[Login]
  B --> C{Credentials valid?}
  C -- No --> B
  C -- Yes --> D{Role}
  D -- Reporter --> E[Reporter dashboard]
  E --> F[Submit complaint]
  F --> G{Form valid?}
  G -- No --> F
  G -- Yes --> H[Save ticket and generate ID]
  H --> I[Assign department and record history]
  I --> J[Track complaint]
  D -- Officer --> K[Department queue]
  K --> L[Review and update status]
  L --> M{Outcome}
  M -- In Progress --> K
  M -- Resolved --> N[Record resolution]
  M -- Rejected --> O[Record rejection reason]
  N --> P[Reporter feedback]
  O --> Q[Reporter views reason]
  D -- Admin --> R[Manage assignments and analytics]
```

3.3 Key alternate paths

- Invalid form: preserve safe field values, show field-level errors and do not create a ticket.
- Image upload failure: do not silently report success; either retry or reject the submission without creating an incomplete ticket.
- Unauthorized ticket access: return a safe 403 or 404 response without revealing whether another user's ticket exists.
- Invalid status transition: reject the update and preserve the current state.
- Database failure: roll back the transaction and show a non-sensitive retry message.

4. UI/UX Design Document — Apple-inspired principles

Design direction: borrow general principles associated with Apple's Human Interface Guidelines—clarity, deference to content, consistency, direct manipulation, accessibility and thoughtful feedback. This is a project-specific design brief, not an official Apple document and not a copy of Apple's proprietary assets.

4.1 Design principles

Principle	Application in this product
Clarity	Use plain language: “New complaint”, “In progress”, “Resolved”; avoid internal technical terms.
Hierarchy	Ticket ID and status are prominent; secondary metadata is visually quieter.
Consistency	Reuse the same navigation, buttons, spacing, status labels and form patterns.
Feedback	Show loading, success, validation and failure states close to the action.
User control	Allow users to review their submission and understand what happens next.
Accessibility	Labels, keyboard operation, focus states, contrast, non-color status cues and readable text.
Responsive behavior	Prioritize a one-column mobile layout; expand to multi-column dashboards on desktop.

4.2 Visual system

Token	Recommendation
Primary text	Deep navy / near-black (#14243A)
Accent	Calm blue-teal (#176B87) for primary actions and links
Page background	Very light neutral (#F6F8FA)
Surface	White cards with subtle borders; avoid excessive shadows
Success	Green paired with a text label and check icon
Warning	Amber paired with explanatory text
Error	Red paired with a clear error message; never rely on color alone
Typography	System font stack: -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif
Spacing	Use a consistent 4/8-point spacing rhythm
Corners	Moderately rounded cards and inputs; consistent radius scale

4.3 Navigation and information architecture

Reporter navigation	Officer navigation	Admin navigation
Dashboard	Department queue	Overview

New complaint	Filters / assigned tickets	All complaints
My complaints	Complaint detail	Assignments
Notifications (optional)	Resolution history	Departments and users
Profile / sign out	Profile / sign out	Analytics / sign out

4.4 Screen specifications

Screen	Layout and components	Important states
Login	Compact centered form, university identity, email, password, submit	Invalid credentials, loading, disabled submit
Reporter dashboard	Welcome header, summary cards, New Complaint primary action, recent tickets	First-use empty state, loading, error
New complaint	Title, category, location, description, optional image, submit	Field errors, file preview, upload progress, success
Complaint detail	Ticket ID, status, category, location, image, timeline, department, remarks	Loading, not found/unauthorized, rejected reason, resolved feedback
Officer queue	Search, status/priority filters, sortable table or mobile cards	No assignments, no filter matches, update failure
Admin analytics	Summary cards, category chart, status chart, trend chart, department table	No data, loading, chart error

4.5 Component behavior

- Primary button: one dominant action per screen, e.g. Submit complaint.
- Status badge: show both label and consistent icon/shape; keep labels stable.
- Form errors: place near the relevant field and summarize errors at the top for long forms.
- Image attachment: show filename, preview where safe, remove action and upload limit.
- Timeline: newest/oldest ordering must be consistent and timestamps human-readable.
- Destructive or consequential actions: require confirmation when appropriate and explain the effect.
- Tables: support pagination and mobile adaptation; do not shrink text until it becomes unreadable.

4.6 Accessibility and responsive requirements

- All controls must be reachable and operable with a keyboard.
- Every input has a visible label; placeholders are not substitutes for labels.
- Provide visible focus indicators and semantic heading order.
- Use meaningful alt text for informative images and empty alt text for decoration.
- Meet WCAG AA contrast targets where practical; never convey status by color alone.
- Test layouts at approximately 360 px mobile width, tablet and desktop widths.
- Respect reduced-motion preferences; animations must not block tasks.

4.7 Wireframe descriptions

Reporter dashboard: top navigation with product name and profile; page title and New Complaint action; four compact summary cards; recent complaint list with ticket ID, title, category, date and status; empty state with a direct call to action.

Complaint form: single-column form on mobile and constrained-width form on desktop; required fields grouped first; attachment panel below description; clear submission button and privacy hint; success screen presents ticket ID with copy action.

Officer queue: compact filter row above a table; status and priority remain readable; opening a row shows detail panel/page with timeline and authorized status controls.

Admin analytics: summary metrics first, charts second, detailed department breakdown third; date range/filter

controls should affect every chart consistently.

5. Backend and Database Schema

5.1 Entity overview

Table	Purpose	Important relationships
users	Accounts and role identity	Referenced by complaints, history and feedback audit context
departments	Campus service teams	One department has many complaints
categories	Configurable issue categories and default routing	Category maps to default department
complaints	Canonical ticket record	Reporter, department and optional assigned officer
complaint_history	Append-only status/action history	Many events per complaint
feedback	Reporter rating after resolution	At most one feedback record per complaint in MVP

5.2 Recommended schema rules

- Use BIGINT identity primary keys for internal relations and a separate unique ticket_id for user-facing tickets.
- Use TIMESTAMPTZ in PostgreSQL and store timestamps in UTC; format for the user's locale in the UI.
- Use foreign keys for relationships and CHECK constraints for finite role/status/priority values.
- Use ON DELETE RESTRICT or soft-deletion for users who are referenced by tickets; preserve audit integrity.
- Add indexes for reporter_id, department_id + status, assigned_to, created_at and ticket_id.
- Make history append-only at the application level; do not overwrite earlier events when a status changes.
- Store image storage keys/paths, not base64 image data, in the complaints table.

5.3 PostgreSQL starter DDL

```
CREATE TABLE users (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(120) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  role VARCHAR(20) NOT NULL CHECK
    (role IN ('student','staff','officer','admin')),
  department_id BIGINT,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE departments (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE,
  is_active BOOLEAN NOT NULL DEFAULT TRUE
);

ALTER TABLE users ADD CONSTRAINT fk_users_department
  FOREIGN KEY (department_id) REFERENCES departments(id);

CREATE TABLE categories (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(80) NOT NULL UNIQUE,
  default_department_id BIGINT REFERENCES departments(id),
  is_active BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE complaints (
```

```
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  ticket_id VARCHAR(32) UNIQUE,
  reporter_id BIGINT NOT NULL REFERENCES users(id),
  category_id BIGINT NOT NULL REFERENCES categories(id),
  department_id BIGINT REFERENCES departments(id),
  assigned_to BIGINT REFERENCES users(id),
  title VARCHAR(200) NOT NULL,
  description TEXT NOT NULL,
  location VARCHAR(200) NOT NULL,
  image_key TEXT,
  priority VARCHAR(20) NOT NULL DEFAULT 'Normal'
    CHECK (priority IN ('Low','Normal','High','Urgent')),
  status VARCHAR(20) NOT NULL DEFAULT 'Submitted'
    CHECK (status IN ('Submitted','In Progress','Resolved','Rejected')),
  rejection_reason TEXT,
  resolution_notes TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  resolved_at TIMESTAMPTZ,
  CHECK (status <> 'Rejected' OR rejection_reason IS NOT NULL),
  CHECK (status <> 'Resolved' OR resolved_at IS NOT NULL)
);

CREATE TABLE complaint_history (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  complaint_id BIGINT NOT NULL REFERENCES complaints(id),
  actor_id BIGINT NOT NULL REFERENCES users(id),
  action VARCHAR(40) NOT NULL,
  old_status VARCHAR(20),
  new_status VARCHAR(20),
  remarks TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE feedback (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  complaint_id BIGINT NOT NULL UNIQUE REFERENCES complaints(id),
  rating SMALLINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
  comments TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_complaints_reporter_created
  ON complaints(reporter_id, created_at DESC);
CREATE INDEX idx_complaints_department_status
  ON complaints(department_id, status);
CREATE INDEX idx_complaints_assigned_to
  ON complaints(assigned_to);
CREATE INDEX idx_complaint_history_ticket
  ON complaint_history(complaint_id, created_at);
```

Implementation note: the DDL above includes a users.department_id foreign key added after departments is created to avoid circular table-order issues. Ticket ID generation and updated_at maintenance must be implemented in a transaction, trigger or service layer. If a status constraint is too strict during intermediate updates, enforce the final invariant in the service layer and test it.

5.4 Ticket ID generation

Recommended approach: insert the complaint to obtain its database-generated ID, format a public ID such as CMP-2026-00042, then persist it in the same database transaction. For stronger guarantees under concurrency, use a dedicated sequence or database trigger and retain a UNIQUE constraint on ticket_id. Avoid a count(*) + 1 approach because concurrent submissions can collide.

5.5 Analytics definitions

Metric	Definition
Total complaints	Count of complaint records in selected period
Open complaints	Submitted + In Progress
Resolution rate	Resolved count / all complaints created in selected period; document denominator
Average resolution time	Average(resolved_at - created_at) for resolved complaints only
Average feedback	Average rating across feedback records
Overdue complaints	Open complaints older than a configured target; target is a product decision

6. API / Route Contract

For a server-rendered Flask MVP, HTML form routes are sufficient. If the frontend later becomes a separate SPA, expose equivalent JSON APIs. All state-changing requests require authentication, CSRF protection and server-side authorization.

Method	Route	Access	Purpose
GET/POST	/login	Public	Login form and authentication
POST	/logout	Authenticated	End session
GET	/dashboard	Authenticated	Redirect/render dashboard by role
GET/POST	/complaints/new	Reporter	Display and create complaint
GET	/complaints	Authenticated	List only permitted complaints
GET	/complaints/<ticket_id>	Authorized viewer	Complaint detail and history
POST	/complaints/<ticket_id>/assign	Admin	Assign department/officer
POST	/complaints/<ticket_id>/status	Officer/admin	Update status and history
POST	/complaints/<ticket_id>/feedback	Reporter of resolved ticket	Save feedback once
GET	/admin/analytics	Admin	Read analytics

6.1 Error handling conventions

- 400: malformed or invalid request; return field-level messages for forms.
- 401: not authenticated where JSON APIs are used; otherwise redirect to login.
- 403/404: unauthorized or unavailable complaint, without leaking private record existence.
- 409: conflicting update or duplicate feedback where JSON APIs are used.
- 413: upload exceeds configured size.
- 500: generic user-facing error plus server-side diagnostic log; never show stack traces in production.

7. Implementation Plan

Phase	Tasks	Exit criteria
0. Decisions and setup	Confirm stack, roles, categories, image limit, status rules; create repo and virtual environment.	App starts locally; team agrees on schema and route contract.
1. Database foundation	Create models, migrations, departments/categories seed data and demo accounts.	Fresh database can be initialized from migrations.
2. Authentication and layout	Login/logout, password hashing, role guards, base template and responsive navigation.	Each role lands on the correct page; protected routes deny unauthorized access.
3. Complaint creation	Form validation, ticket ID, category	A valid complaint persists and can be

	routing, image storage and initial history.	reopened by ticket ID.
4. Tracking and officer workflow	Reporter list/detail, department queue, assignment, status transitions and remarks.	Officer update appears in reporter timeline and audit history.
5. Feedback and analytics	Feedback validation, aggregate queries, Chart.js visualizations.	Metrics match known seed data and calculations are documented.
6. Hardening and test	Authorization tests, upload tests, mobile UI, error handling and database rollback checks.	Critical test cases pass; no known cross-user access flaw.
7. Demo and deployment	Seed demo data, configure environment, deploy backend/database/storage, rehearse scenarios.	End-to-end demo works from a clean browser session.

7.1 Suggested 7-day schedule

Day	Focus	Deliverable
1	Setup and schema	Repository, database migrations, seed data
2	Authentication and layout	Login, role checks, shared UI shell
3	Complaint submission	Form, image validation, ticket ID, routing
4	Tracking and workflow	Ticket list/detail, officer queue, status history
5	Admin and feedback	Feedback form, summary queries and charts
6	Testing and polish	Authorization tests, responsive fixes, error states
7	Integration and demo	Deployment, demo data, final walkthrough

7.2 Team split for three members

Member	Ownership	Integration agreement
Member A — Frontend	Jinja templates, CSS, client-side validation, charts	Use agreed route names and field names; do not simulate backend results.
Member B — Backend	Flask routes, authentication, validation, assignment and status workflow	Own service-layer rules and authorization tests.
Member C — Data / QA / integration	SQLAlchemy models/migrations, seed data, image storage integration, analytics tests and deployment support	Keep schema migrations and seed scripts in Git; verify all roles end-to-end.

7.3 Git workflow

1. Create main and feature branches; protect main from unreviewed direct changes.
2. Agree on models, form fields and route contracts before parallel implementation.
3. Use small commits and pull requests; run tests before merging.
4. Keep .env, uploads containing real personal data and production secrets out of Git.
5. Integrate the submission flow early instead of waiting for all pages to be finished.

8. Acceptance Criteria and Test Plan

Test ID	Scenario	Expected outcome
T-01	Valid login	User session created and role-specific dashboard shown.
T-02	Invalid password	Login rejected without revealing which credential was wrong.
T-03	Valid complaint	One record created with unique ticket ID and initial history.

T-04	Missing required title/location	No record created; clear validation shown.
T-05	Unsupported/oversized image	Upload rejected safely; no false success.
T-06	Wi-Fi category	Default department is IT Support.
T-07	Student guesses another ticket ID	Private data is not returned.
T-08	Officer updates unrelated department ticket	Request denied.
T-09	Reject without reason	Update blocked.
T-10	Resolve ticket	Resolution notes and resolved_at are saved; history appended.
T-11	Submit duplicate feedback	Second feedback record blocked.
T-12	Analytics with known seed data	Counts and averages match independently calculated expected values.
T-13	Database error during submission	Transaction rolls back; no incomplete ticket remains.
T-14	Mobile viewport	Form, ticket detail and primary actions remain usable.

8.1 Demo walkthrough

6. Log in as a student and submit a Wi-Fi complaint with a sample image.
7. Show the generated ticket ID and default IT Support assignment.
8. Open My Complaints and show the Submitted state.
9. Log out and log in as an IT officer; open the department queue.
10. Move the complaint to In Progress and add a remark.
11. Resolve the complaint with resolution notes.
12. Return to the student account and show the status timeline; submit a rating.
13. Log in as admin and show updated complaint counts and resolution metrics.

9. Risks, Assumptions and Decisions

Topic	Current assumption / risk	Decision needed
Identity	University email or university ID is used for login.	Confirm whether registration is open or admin-provisioned.
Anonymous grievances	Not included in MVP; identity is linked to a ticket.	Decide if sensitive/anonymous reporting is required.
Urgent complaints	Priority is a label, not an emergency response service.	Define campus escalation policy before production use.
SLA / overdue	No official response-time targets supplied.	Set targets by category/department if needed.
Images	Attachments may contain identifiable people or sensitive details.	Define private storage, access, retention and deletion policy.
Rejected complaints	A reason is required; reopening is not in MVP.	Decide whether an appeal or reopen workflow is required.
Notifications	In-app updates are optional; email is a later enhancement.	Select email provider only if time permits.
Production readiness	Hackathon scope is not a full institutional deployment.	Obtain security, privacy, backup and operations review before real use.

Reference basis

Source document: REVA Weekend Hackathon, “Problem Statement #3: Smart Campus Complaint and Grievance Management System,” supplied document pages 3–4. The source specifies complaint submission with images, categories, automatic IDs, department assignment, status tracking, feedback/resolution-time analytics, and the prototype flow Login → Submit Complaint → Generate Ticket → Assign Department → Update Status → Close

Ticket.

Design note: Apple-inspired wording in this document refers to broad interface design principles, not a claim of official Apple authorship or endorsement. Consult Apple's current Human Interface Guidelines directly when finalizing platform-specific patterns.